Project 1: Algorithmic Time-Series Segmentation and Condition Analysis Using Water Pump RUL Dataset

Owen Mallon
CmpSc 463 – Dsgn Analysis Algs

1. Project Overview

   The goal of this project is to develop an algorithm-driven system to segment, cluster, and analyze time-series sensor data from a real water-pump machine.
   The system will use core algorithmic techniques—Divide-and-Conquer segmentation, Closest Pair, and Maximum Subarray—to characterize machine behavior and relate the results to four machine-health categories derived from RUL (Remaining Useful Life).

   This project does not use machine learning libraries. The emphasis is on algorithm design, analysis, and explanation.

   This overview was pulled directly from the canvas project assignment

2. Installation and Usage Instructions

   The required libraries for this program are as follows:
   - NumPy
   - Matplotlib
   - Pandas
   - Random
   - Collections

   Depending on your IDE, you might be able to install the required packages right from IDE application, however using console to manually install libraries might be required.

   To use program, run “Project1_Main”, program will first output visual display of task 1, after closing that popup, console will print out results from Task 2 and 3 respectively.

3. Code Structure Explanation

   The file structure is split into each respective task, each having its own .py file, imported into main for usage. It was designed this way so that each task was separated and alone allowing for better code readability and understanding, as well

as making bug testing task specific if need be. Everything that is used together is bundled together, instead of needing to scroll through one long .py file with every function put together. Also included within the general file structure is both the dataset and this report, all uploaded to GitHub/

4. Algorithm Descriptions
    a. Task 1: Divide-and-Conquer Segmentation

        The goal of Task 1 was to: “Implement a recursive segmentation algorithm on one selected sensor signal (e.g., sensor_00) or an engineered composite signal (e.g., average of all sensors)”. For my algorithm, the computer randomly selects 10 total sensors to work on. When the algorithm is first called, the computer passes all the data of the specific sensor, which then calculates the total variance of that sensor data. Given a predetermined variance threshold, if the current segment variance is greater than the threshold, the program will then begin to recursively split the segment into halves. The program will continue this recursive splitting until each segment's variance is less than the given threshold, otherwise known as a “stable segment”. It is fully possible for a sensor to be marked stable on its original function call, however it is very unlikely. After the segment is split up and contains only stable segments, the total number of segments is called the “Segmentation Complexity Score”.

    b. Task 2: Divide-and-Conquer Clustering of Segments

        The goal of Task 2 was to: “Design a recursive top-down clustering algorithm with a designated number of clusters”. To complete this task, the program handles all 10,000 time-instances of each sensor at once. The algorithm then recursively splits the dataset into halves based off of the current median, until a predetermined number of clusters exist, which for this project was 4. For us, the algorithm recursively splits the datasets twice, giving us 4 total clusters, each split at the median of the parent cluster. After the required number of clusters was produced, the program then determines the most present RUL category of each and is then used to label the RUL of that specific cluster allowing us to analyze any patterns.

    c. Task 3: Maximum Subarray (Kadane)

The goal of Task 3 was to: “Implement Kadane algorithm to find the maximum subset sum and corresponding segment for a given time series”. This was done to determine which periods of time a sensor went through the most derivation between points in time. To complete this task, the algorithm would scan through all time-instance data points from a sensor and using Kadanes’ algorithm, it determines the largest contiguous portion of a sequence whose elements produce the largest possible sum. Overall, this can be used as another way to determine the RUL of a sensor, as based on the largest sum of derivations you can see the stability of the array due to the derivation during the time period calucated as the maximum subarray.

5. Dataset Description

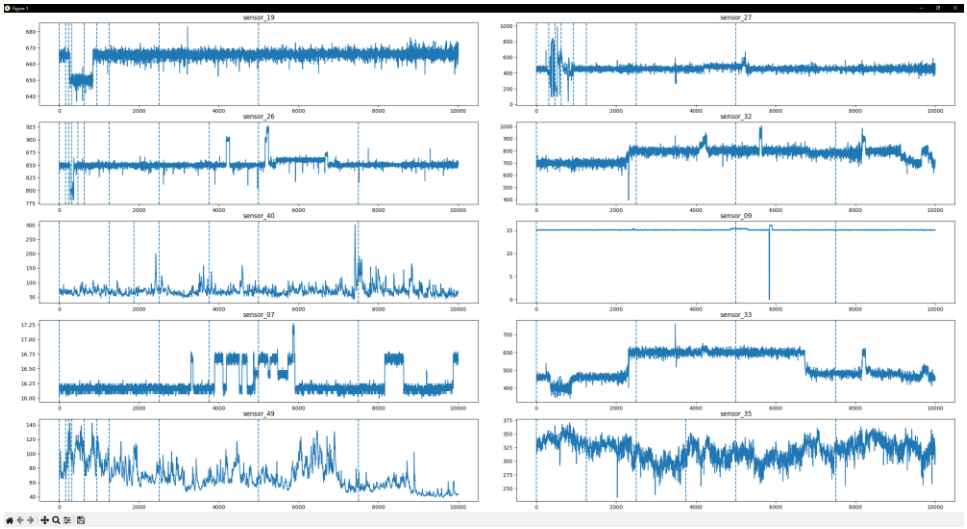
All the data downloaded from the linked site was used; however, only the first 10,000 rows were collected, and some redundant/unnecessary columns were removed for ease of use and memory saving, such as: “Unnamed: 0”. Additionally, out of the 52 expected sensors there were 2 missing sensors, sensors 15 and 50, making the dataset incomplete. The cause of this issue is unknown, as the dataset was used as is from download, meaning the data from the website itself was incomplete from the beginning.

Additionally, before any tasks function was run, there was some basic maintenance and verification tasks completed on the dataset before it was used within the tasks. Firstly, the program confirms the number of sensors present and created the Remaining Useful Life (RUL) quartiles and then sorts each sensor respectively based on the following parameters:

Condition	Definition
Extremely Low RUL	RUL < Q10
Moderately Low RUL	Q10<= RUL< Q50
Moderately High RUL	Q50<= RUL< Q90
Extremely High RUL	RUL >= Q90

Execution Results

a. Task 1


sensor_19
sensor_27
sensor_26
sensor_32
sensor_40
sensor_09
sensor_07
sensor_33
sensor_49
sensor_35

b. Task 2

```
Segmentation Complexity Scores:
sensor_19 : 9
sensor_27 : 8
sensor_26 : 11
sensor_32 : 4
sensor_40 : 7
sensor_09 : 3
sensor_07 : 5
sensor_33 : 4
sensor_49 : 10
sensor_35 : 6

Cluster 0
Cluster Size: 2500
Majority Class: ('Moderately High', 1435)

Cluster 1
Cluster Size: 2500
Majority Class: ('Moderately Low', 1631)

Cluster 2
Cluster Size: 2500
Majority Class: ('Moderately Low', 1392)

Cluster 3
Cluster Size: 2500
Majority Class: ('Moderately High', 1488)
```

c. Task 3

```
Sensor: sensor_00
Max Deviation Interval: 62 - 4830
Deviation Score: 9.493931137414075
RUL Category Distribution:
rul_category
Moderately High    3830
Extremely High      938
Name: count, dtype: int64

Sensor: sensor_01
Max Deviation Interval: 1 - 2038
Deviation Score: 184.00498048484283
RUL Category Distribution:
rul_category
Moderately High    1038
Extremely High      999
Name: count, dtype: int64

Sensor: sensor_02
Max Deviation Interval: 8990 - 9825
Deviation Score: 19.57380504361334
RUL Category Distribution:
rul_category
Extremely Low     825
Moderately Low     10
Name: count, dtype: int64

Sensor: sensor_03
Max Deviation Interval: 5217 - 9488
Deviation Score: 32.33858125479592
RUL Category Distribution:
rul_category
Moderately Low    3783
Extremely Low      488
Name: count, dtype: int64
```

6. Discussion and Conclusions

In terms of coding, this project was a massive jump in previously built solo projects. This required several different understandings and required some new skills to be learned over a short period of time. Additionally, it required a new level of thinking to better understand the overall results gathered by the data worked with. Those thoughts are as follows below, which are all questions asked within each task that weren't answered above:

In general, the complexity score of a sensor can be directly tied to the RUL of said sensor. When a sensor has high variance, it will then in turn have more segments once segmentation is finished, giving the determination that a sensor is, as a result, generally unstable. Overall, the number of segments generated from each sensor can be used to represent the overall stability, as well as the RUL, of said sensor.

Additionally, the outputs of task 1 can be used to determine the overall temporal dynamics of a specific sensor. Overall, the quantity of segments can be used to analyze how a sensor operates over time, with less segments being representative of a smoother and more consistent sensor, and the opposite being more unstable and constantly changing over shorter periods.

For task 2, each cluster maps to a specific RUL class by using the mode of the cluster to determine the general RUL of the cluster. Instead of taking the average and median, it uses simply the most dominant class. Understandably so it is clear to see that there is now extreme outlier within this dataset, and the most present is somewhere within the moderate range of RULs, as while it's determined by mode, it is still very much based off of quartiles, giving it some similarities.

For task 3, after applying the max subarray algorithm, we can determine the overall health of each sensor and if there are any indicators of degradation in the results. By using the interval of which the max derivation occurred, we can determine if the sensor was showing either early signs of degradation, or very late signs. This can be described as follows:

When deviation occurs	Interpretation
Extremely High RUL	Very early indicator
Moderately High RUL	Early warning signal
Moderately Low RUL	Mid-stage degradation signal
Extremely Low RUL	Late failure signal

Some possible improvements for this project and code would be implementing better output for the results, as except for task 1, the code uses the IDE console output to print any results. This is something that would build off task 1's existing GUI display, however that would require creating a whole new main menu, as well as custom outputs for task 2 and 3. Additionally, I realized way too late but my documentation throughout the code was very lacking and could definitly use some improvement before the next project is to be completed. I need to add better method explanations in the actualy code files instead of just within the report.

Some limitations within this project were simply understandings that needed to be cleared up within general coding. When it comes to recursion, I understand previously taught algorithms, but attempting to create my own always comes across a lot more difficult than I think it should be. This is definitely something I will work on outside of class and in my own time. Lastly, one limitation that occurs when working with large datasets is how my computer handles it. While my computer had no major issues, it wouldn't be too far out there to see some issues appear within initializing, reading, and handling the data.